

FastAPI End-to-End Enterprise Deployment

Infrastructure on AWS ECS Fargate with Automated GitHub Actions CI/CD Pipeline

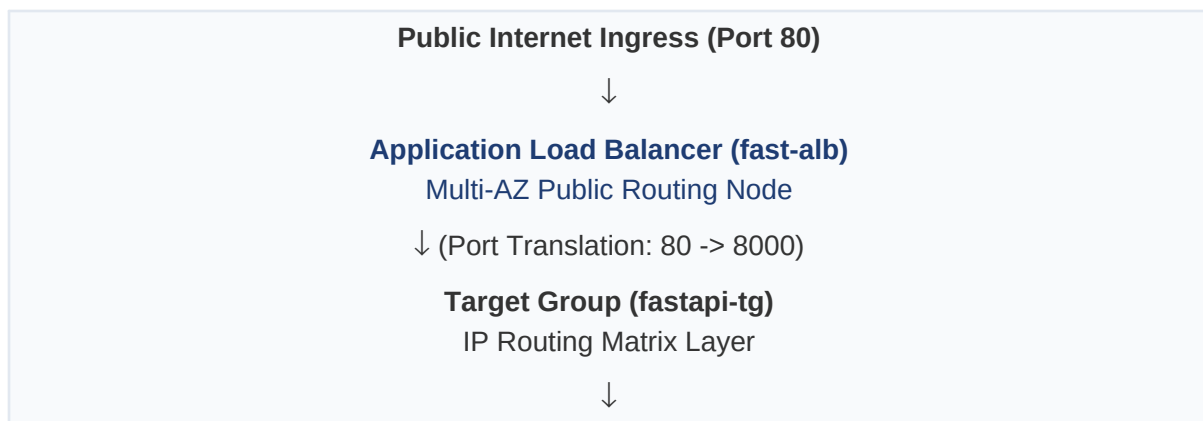
1. Executive Summary

This comprehensive technical document outlines the architecture, deployment configurations, and automated pipeline established for the FastAPI Python application (Project Kanchan). The final architecture successfully hosts the application in a highly available, completely containerized, serverless cloud environment utilizing AWS ECS Fargate, abstracting away server maintenance while implementing a strict, automated zero-downtime continuous integration and continuous delivery (CI/CD) cycle through GitHub Actions.

2. Infrastructure & Architectural Blueprint

The system network topology is engineered following cloud security isolation boundaries:

- **Virtual Private Cloud (VPC):** Isolated space bound to CIDR block 10.0.0.0/16.
- **Availability Zones:** Multi-AZ infrastructure mapping us-east-1a and us-east-1b to insulate from edge runtime failure spikes.
- **Public Subnets:** Hosts the public-facing Application Load Balancer (ALB) acting as the network's gatekeeper.
- **Compute Execution Layer:** Fargate serverless containers executing workloads natively in public/private layers bound to secure security groups.
- **Traffic Redirection Model:** Users issue standard HTTP traffic targeting port 80. The ALB dynamically translates and forwards requests straight down to the backend containers listening strictly on internal app port 8000.



AWS ECS Fargate Cluster (api-cluster)
[Task Instance 1 (us-east-1a)] | [Task Instance 2 (us-east-1b)]
Running Container: *main* listening on port 8000

3. Summary of Core Milestones Completed

Phase 1: Networking Framework & Isolation

Configured VPC networking bounds, subnet segregation tables, ACL matrices, and inbound security rules chaining the application layer to intercept traffic solely if it is processed by the ALB proxy router.

Phase 2: Continuous Integration Engine & Containerization

Engineered a functional multi-stage Dockerfile packaging the FastAPI application inside python:3.11-slim runtimes. Configured GitHub Actions to automatically run linters (flake8) and execute tests (pytest) prior to any container output build operations.

Phase 3: Secure Registry & AWS Identity Provisioning

Constructed a continuous delivery image deployment bridge via Docker Hub. Implemented secure cloud validation profiles using OpenID Connect (OIDC) protocols, eliminating the need to save high-risk root access keys inside GitHub.

Phase 4: ECS Orchestration & Zero-Downtime Releases

Created an AWS ECS Fargate Cluster mapping an active task definition schema running container identity 'main' on internal port 8000. Connected the execution lifecycle hook to GitHub workflows to force a zero-downtime rolling update whenever code changes push directly into the main repository branch.

4. Key Infrastructure Configuration Files

Below is the structural layout of the automated GitHub Actions CI/CD configuration file:

```
File Path: .github/workflows/ci.yml
name: FastAPI CI/CD Pipeline
on:
  push: branches: [ main ]
jobs:
  test-and-lint: ... [Runs flake8 & pytest via working-directory: ./project-api]
  build-and-push:
    needs: test-and-lint
    permissions: id-token: write, contents: read
    steps:
      - Login to Docker Hub (vars.DOCKERHUB_USERNAME / secrets.DOCKERHUB_TOKEN)
      - Build & Push image with latest & unique commit git SHA tags
      - Configure AWS credentials via OIDC role-to-assume (secrets.AWS_ROLE_ARN)
      - Download Active Task Definition
      - Mutate JSON Task Template via container-name: main
      - Deploy modified Definition to cluster: api-cluster, service:
fastapi-task-service-44vbmspx
```

5. Financial Management & Maintenance Matrix

An analysis of hourly base resource metrics inside the us-east-1 region yields the following operational expense tracking layout:

AWS Component	Resource Profile	Estimated Monthly Cost
Application Load Balancer	Base fee per processing cluster node	\$16.43 / mo
AWS Fargate Compute	Serverless Memory/CPU slice (1 Task allocation)	\$1.50 / mo
CloudWatch Logs & Metrics	Data collection ingest footprint	\$1.00 - \$3.00 / mo
Total Projected Budget	Single running task deployment footprint	\$22.93 - \$24.93 / mo

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.